LAB 1

How would you design a library management system using object-oriented programming, and what classes and objects would be involved? How would you use composition to model the relationship between the Loan object and the User and Book objects in the system?

Solution

To design a library management system using object-oriented programming, we would need several classes such as User, Book, Account, Librarian, and Loan. Each class has its own set of properties and methods that would be needed to implement the system. For example, the User class might have properties such as user ID, name, and email address, as well as methods for borrowing and returning books, viewing their account information, and managing their reading lists.

To model the relationship between the Loan object and the User and Book objects, we would use composition. The Loan object should be composed of both the User and Book objects because the loan cannot exist independently without both objects. This means that when the User or Book object is destroyed, the Loan object should also be destroyed.

Overall, by using object-oriented programming concepts such as classes, objects, and composition, we can create a library management system that is robust and flexible, allowing users and librarians to efficiently manage the library's collection and borrow books

LAB 2

Suppose you are working on a project to create a system for a zoo that will keep track of the animals in their exhibits. You have a Mammal class and two subclasses, Lion and Giraffe, each with a method makeSound() that returns the sound the animal makes. You

need to write a method that will iterate over a list of Mammal objects and print out the sound each animal makes.

How would you use polymorphism to write this method, and what advantages does this approach offer over writing separate methods for each type of animal?

LAB 3

What is a problem that can be addressed using good programming practices in OOP, and how can these practices improve the quality and maintainability of the codebase? Specifically, how can practices such as modular and reusable code, naming conventions, proper code commenting, and thorough testing help developers maintain code quality, reduce maintenance costs, and improve system reliability over time?

Solution

The problem of maintaining code quality and reducing the cost of maintenance can be effectively addressed by following good programming practices in OOP. Writing modular and reusable code enables developers to break down complex systems into smaller, more manageable components, making the code easier to understand, maintain, and extend. Adhering to naming conventions and commenting code properly helps improve code readability and clarity, reducing the time and effort required for maintenance and modification. Thorough testing of the codebase can also help catch and fix errors and bugs before they cause significant problems down the line. Overall, by following these practices, developers can create more reliable and maintainable software systems that are easier to modify and update over time.

LAB 4

You are tasked with creating a program that models a simple bank account management system using object-oriented programming concepts. The system should be able to create new accounts, deposit and withdraw funds, and provide the current balance for a given account. However, the system must also follow good programming practices to ensure maintainability and scalability in the future.

Solution

To model the bank account management system, we can create a BankAccount class that encapsulates all relevant data and functions. The class will have properties such as accountNumber, accountHolderName, accountType, and balance. It will also have methods such as createNewAccount(), depositFunds(), withdrawFunds(), and getBalance(). To ensure good programming practices, we will use proper naming conventions, encapsulation to hide the implementation details, and avoid the use of global variables. We will also add thorough comments throughout the code and write unit tests to ensure that the system behaves as expected under a variety of conditions.

LAB 5

Make a basic and simple and basic interface in Visual Studio.

Solution

Step 1: Create a New Project

Open Visual Studio.

- Click on "Create a new project".
- In the project template list, select "Windows Forms App (.NET)" if you are using .NET Core or "Windows Forms App (.NET Framework)" if you are using .NET Framework.

- Give your project a name and specify the location where you want to save it.

Click "Create".

Step 2: Design the User Interface

- Once your project is created, you will see a form (Form 1) in the Designer view.
- From the Toolbox (usually on the left side of the screen), you can drag and drop controls onto the form.
- Add some basic controls such as:
- Label: Drag and drop a label onto the form to display some text.
- TextBox: Add a text box to allow the user to enter text.
- Button: Add a button that the user can click.
- Position the controls as you desire by dragging them around the form.

Step 3: Customize Controls

- Select a control on the form to view its properties in the Properties pane (usually on the right side of the screen).
- Customize the properties of each control (e.g., text, size, color) as desired.

LAB 6

Assuming you have a Git repository with 100 files in it and you have made changes to all the files, how can you push these changes to a remote repository named "origin" on a branch named "develop" while excluding some files from being pushed?

Solution

- git add . - This adds all changes in the current directory and subdirectories to the staging area for the next commit.
- git commit -m "Commit message" - This commits changes to the local repository with a descriptive commit message.
- git pull origin develop - This downloads changes from the remote repository's "develop" branch and merges them with the local repository.
- git reset HEAD file1 file2 file3 - This removes the specified files (file1, file2, and file3 in this example) from the staging area. Replace these file names with the names of the files you want to exclude from the push.
- git commit --amend - This opens the commit message in the default text editor so that you can update the message or add details about the changes you made.
- git push origin develop - This uploads local changes to the remote repository's "develop" branch.

LAB 7

Set up any library management system in Jira

Solution

Step 1:

Create a New Project
Log in to Jira: Open Jira and log in with your credentials.
Create a New Project:
Click on the Projects drop-down in the top navigation bar.
Select Create Project.
Choose a project template. For a library management system, you might use the Kanban or Scrum templates for better visualization of tasks. For this example, we'll use Kanban.
Name your project, e.g., "Library Management System".
Click Create.

Step 2:

Configure Project Settings
Project Settings:
Navigate to your new project.
Click on Project Settings (usually found at the bottom of the left sidebar).
Issue Types:
Go to Issue Types.
Ensure you have issue types such as Task, Sub-task, Bug, and Epic.
Add custom issue types if needed, such as Book, Member, Loan, and Return.
Fields:
Go to Fields.
Add custom fields specific to library management, such as Book Title, Author, ISBN, Member ID, Loan Date, and Return Date.
Screens:
Customize screens for each issue type to include the necessary fields.
Ensure that all relevant fields are available when creating, editing, and viewing issues.

Step 3:

Create Workflows
Workflows:
Go to Workflows in Project Settings.
Create or modify workflows for each issue type. For example:
Book Workflow: New → Catalogued → On Loan → Returned → Archived.
Loan Workflow: Requested → Approved → On Loan → Returned.
Ensure transitions between statuses make sense for your library processes.
Assign Workflows to Issue Types:
Navigate to Workflow Schemes.
Assign the appropriate workflows to each issue type.

Step 4:

Add Automation Rules
Automation:
Go to Project Settings → Automation.
Set up rules to automate common tasks, such as:
Transition a book to "On Loan" when a loan is approved.
Send notifications to members when their loan is due soon.

Step 5:

Test the System
Run a Test Cycle:
Create test data and run through typical scenarios to ensure everything works as expected.
Adjust workflows, fields, and screens as necessary based on test results.
Step 9: Training and Documentation
Train Staff:
Provide training sessions for library staff on how to use the new Jira system.
Create user manuals or documentation to help with onboarding and troubleshooting.

LAB 8

Create an accounts management system in Asana

Solution

Step 1: Create a New Project

Log in to Asana: Open Asana and log in with your credentials.
Create a New Project:
Click on the + New button in the top bar.
Select Project.
Choose a template or start with a blank project. For an accounts management system, starting with a blank project is recommended for customization.
Name your project, e.g., "Accounts Management System".
Choose the layout: List or Board. For detailed tracking, List might be more suitable.

Step 2: Define Sections and Custom Fields

Create Sections:
Within your project, create sections to categorize tasks. Examples of sections might include:
New Accounts
Active Accounts
Inactive Accounts
Follow-ups
Billing
Add Custom Fields:
Click on Customize in the top right corner.
Select Add Custom Field.

Add fields relevant to accounts management, such as:
Account ID
Account Manager
Status (Dropdown: New, Active, Inactive)
Contact Information
Billing Cycle
Last Contact Date
Next Follow-up Date

Step 3: Add Tasks

Create Tasks:
Add tasks under the relevant sections for each account you manage.
Each task can represent an individual account.
Fill in the custom fields for each account to keep all necessary information organized.

Step 4: Set Up Automation

Rules:
Click on Customize and then Add Rule.
Set up rules to automate processes. Examples:
When a task is moved to the "Active Accounts" section, set the Status field to "Active".
Send notifications to the account manager when the "Next Follow-up Date" is approaching.

Step 5: Utilize Asana Features for Efficiency

Assign Tasks:
Assign tasks to team members responsible for managing specific accounts.
Set due dates for follow-ups and billing cycles.
Add Comments and Attachments:
Use the comment section in tasks to add notes about interactions with the account.
Attach relevant documents such as contracts, invoices, and emails.

Lab 9

A Good Design Class Example

A class called “Engine” can be composed within the “Car” class, representing the car’s engine as a separate object. This allows the Car class to leverage the functionality of the Engine class while maintaining a modular and flexible design.

Solution:

```
class Engine {
private:
    std::string engineType;

public:
    Engine(const std::string& engineType) : engineType(engineType) {}

    void start() {
        std::cout << "Engine started. Type: " << engineType << std::endl;
    }

    void stop() {
        std::cout << "Engine stopped. Type: " << engineType << std::endl;
    }
};

class Car {
private:
    std::string make;
    std::string model;
    Engine engine;
```

```
public:
    Car(const std::string& make, const std::string& model, const std::string& engineType)
        : make(make), model(model), engine(engineType) {}

    void start() {
        std::cout << "Starting the car. Make: " << make << ", Model: " << model << std::endl;
        engine.start();
    }

    void stop() {
        std::cout << "Stopping the car. Make: " << make << ", Model: " << model << std::endl;
        engine.stop();
    }
```

LAB 10

Write a code in MISRA C++ coding standard:

- Take three integers.
- Calculate their sum.
- Calculate the average of these numbers.

Solution:

```
#include <iostream>

// Function declarations
int calculateSum(int a, int b, int c);
```

```
float calculateAverage(int sum, int count);

int main() {
    int num1 = 0;
    int num2 = 0;
    int num3 = 0;

    std::cout << "Enter three integers:" << std::endl;

    std::cin >> num1;
    std::cin >> num2;
    std::cin >> num3;

    // Calculate sum
    int sum = calculateSum(num1, num2, num3);

    // Calculate average
    const int count = 3;
    float average = calculateAverage(sum, count);

    // Display results
    std::cout << "Sum: " << sum << std::endl;
    std::cout << "Average: " << average << std::endl;

    return 0;
}
```

```
// Function to calculate sum of three integers
int calculateSum(int a, int b, int c) {
   return a + b + c;
}

// Function to calculate average of sum
float calculateAverage(int sum, int count) {
   return static_cast<float>(sum) / static_cast<float>(count);
}
```

Lab 11 and 12::

- You are required to open:
- This PC
- Open My Document
- Select any from My Document
- Send this file to the Recycle Bin

Lab 13:

- On the left sidebar, select Code > Repository.
- Above the file list, select the branch you want to commit to. If you're not sure, leave master or main. Then select the plus icon () and the New file.
- For the **Filename**, type .gitlab-ci.yml and in the larger window, paste this sample code.

Solution:

```
build-job:
stage: build
script:
- echo "Hello, $GITLAB_USER_LOGIN!"

test-job1:
stage: test
script:
- echo "This job tests something"
```

Lab 14:

The typical UI interaction for an instrumented test. The tester clicks on the target element to verify that the UI displays another element.

Code:

```
// If the Start button is clicked
onView(withText("Start"))
    .perform(click())

// Then display the Hello message
onView(withText("Hello"))
    .check(matches(isDisplayed()))
```

Lab 15:

The program declares class X , which includes one data object, two constructors, one member function, and a destructor. The member function multiplies the data object by 2. The destructor checks the value of the data object and prints an error message if the data object is not between 7 and 10. The main function uses the constructors to create several instances of class X, calls the member function four times (once for each instance of class X), and then deletes the instances of class X.

Solution:

```
#include <iostream.h>
class X
{
public: int i;
     // first X constructor
  X(int ia)
  {
    i = ia;
  };
    // second X constructor
  X(int ib, int jb)
  {
    i = ib + jb;
  };
     // X member function myfunc()
  void myfunc()
  {
    i *= 2;
```

```
  };
    // X destructor
  ~X()
  {
    int id;
    id = i / 2;
    if (id < 7 || id > 10)
    printf("Error - Out of range\n");
  };
};
int main()
{
  X *x1 = new X(7);
  X *x2 = new X(6,2);
  X *x3 = new X(9);
  X *x4 = new X(9,1);
  x1->myfunc();
  x2->myfunc();
  x3->myfunc();
  x4->myfunc();
  delete x1;
  delete x2;
  delete x3;
  delete x4;
  return 0;
}
```